Лабораторная работа №1

Бинарная классификация: логистическая регрессия и k-ближайших соседей

1. Цель работы

Освоение методов бинарной классификации на основе логистической регрессии и метода k ближайших соседей, а также формирование навыков предобработки данных, построения моделей, оценки их качества и сравнительного анализа результатов на реальном финансовом наборе данных.

2. Источник данных

Используется датасет **Taiwanese Bankruptcy Prediction (1999–2009)**.

Допустимые источники:

- UCI Machine Learning Repository
  https://archive.ics.uci.edu/dataset/572/taiwanese+bankruptcy
- Kaggle — Company Bankruptcy Prediction
  https://www.kaggle.com/datasets/fedesoriano/company-bankruptcy-prediction

Файл: `data.csv`

Перед началом работы необходимо убедиться, что:

- число наблюдений — 6819;
- число переменных — 96;
- присутствует переменная `Bankrupt?`.

В отчёте обязательно указать источник скачивания.

3. Описание набора данных

Набор данных содержит:

- 6819 компаний;
- 95 числовых финансовых признаков;
- 1 бинарную зависимую переменную `Bankrupt?`.

Значения переменной:

- 0 — компания не является банкротом;
- 1 — компания является банкротом.

4. Задание

4.1 Первичный анализ и предобработка данных

Необходимо:

1. Загрузить CSV-файл.
2. Определить размерность и структуру данных.
3. Проверить наличие пропусков.
4. При необходимости выполнить:
    - удаление пропусков;
    - либо их обработку.
5. Выполнить нормализацию признаков (обязательно для метода kNN). Допускается использование функции `scale()`.

В отчёте кратко описать выполненные действия.

4.2 Разделение выборки

Разбить данные на:

- обучающую выборку;
- тестовую выборку.

Допускается:

- случайное разбиение;
- ручное разделение.

Рекомендуемая пропорция — 70/30 или 80/20.
Использовать фиксированное значение `set.seed()`.

4.3 Классификация логистической регрессией

Для зависимой переменной `Bankrupt?` и набора факторов:

{X16, X18, X19, X37, X42, X46, X50, X57, X74, X81}

необходимо:

1. Построить модель логистической регрессии с использованием функции:

```
glm(..., family = "binomial")
```

2. Проанализировать результаты `summary()`:
    - коэффициенты модели;
    - значения p-value.
3. Если в модели присутствуют статистически незначимые факторы — устранять их по одному с повторной оценкой модели.
4. С помощью:

```
predict(type = "response")
```

спрогнозировать вероятность банкротства для:

- одного выбранного банкрота;
- одного выбранного небанкрота.

Допускается выбрать объекты вручную либо из тестовой выборки.

5. При желании выполнить дополнительный эксперимент:
    - провести отбор факторов методом включения или исключения;
    - сравнить полученные результаты.

4.4 Классификация методом k ближайших соседей

Необходимо:

1. Выполнить нормализацию признаков.
2. Использовать функцию `knn()` из пакета `class`.
3. Провести бинарную классификацию.
4. Построить матрицу неточностей с помощью функции `CrossTable()` из пакета `gmodels`.
5. Определить количество ложно определённых банкротов (банкрот классифицирован как небанкрот).

Допускается проведение экспериментов при различных значениях k.

5. Требования к отчёту

Отчёт должен содержать:

1. Цель работы.
2. Указание источника датасета.
3. Описание этапов предобработки.
4. Используемый программный код.
5. Матрицы неточностей для обеих моделей.
6. Количество ложно определённых банкротов.
7. Прогноз вероятности для двух выбранных компаний.
8. Сравнительный анализ методов.
9. Итоговые выводы.

Форма представления отчёта:

- PDF/DOCX
  или
- Jupyter Notebook (*.ipynb) с воспроизводимыми вычислениями.

6. Контрольные вопросы

1. В чём принципиальное отличие логистической регрессии от метода kNN?
2. Почему метод kNN требует нормализации признаков?
3. Что означает статистическая значимость коэффициента в логистической регрессии?
4. Как влияет выбор параметра k на границу разделения классов?
5. Что означает ложная классификация банкрота и почему она является критичной ошибкой?
6. В каких случаях логистическая регрессия может демонстрировать более устойчивый результат по сравнению с kNN?

Лабораторная работа №2

Анализ и прогнозирование временных рядов экономических показателей

1. Цель работы

Освоение методов анализа динамики временных рядов и построения краткосрочных прогнозов с использованием линейной регрессии по временной оси и моделей экспоненциального сглаживания (Holt, Holt–Winters) в среде Python.

2. Исходные данные

Используются два набора данных:

1) Российские макроэкономические показатели

Источник:
https://www.kaggle.com/datasets/zavidnikitamahbub/russian-economy-macroeconomic-data-2005-2021

Используется один квартальный показатель (указать конкретный столбец в методичке).

2) Индекс потребительских цен (CPI)

Источник:
https://www.kaggle.com/datasets/tunguz/us-consumer-price-index-and-inflation-cpi

Используется месячный ряд CPI.

Перед началом работы необходимо:

- скачать CSV-файлы,
- загрузить их в Jupyter Notebook / Google Colab,
- проверить период и частоту данных.

3. Используемые библиотеки

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score

from statsmodels.tsa.holtwinters import ExponentialSmoothing
```

4. Задание

4.1 Подготовка данных

1. Загрузить данные с помощью `pandas`.
2. Преобразовать дату в индекс `DatetimeIndex`.

3. Убедиться в корректной частоте (`.asfreq()`).
4. Построить графики временных рядов.
5. Кратко описать:
    - наличие тренда;
    - наличие сезонности;
    - возможные структурные изменения.

4.2 Линейная модель временного ряда

Для выбранного российского макроэкономического показателя:

1. Создать числовой временной индекс.
2. Построить модель линейной регрессии:

```
X = np.arange(len(series)).reshape(-1,1)
y = series.values

model = LinearRegression()
model.fit(X, y)

y_pred = model.predict(X)
r2 = r2_score(y, y_pred)
```

3. Рассчитать:
    - коэффициент детерминации;
    - прогноз на 1 шаг вперёд.

4.3 Модель Хольта (без сезонности)

Для годового или квартального ряда:

```
model_holt = ExponentialSmoothing(series, trend='add', seasonal=None)
fit_holt = model_holt.fit()

forecast_holt = fit_holt.forecast(steps=3)
```

Получить прогноз на заданный период.

4.4 Модель Хольта–Винтерса

Для месячного ряда CPI:

1. Автоматический подбор параметров:

```
model_hw = ExponentialSmoothing(series,
                                trend='add',
                                seasonal='add',
                                seasonal_periods=12)

fit_hw = model_hw.fit()
forecast_hw = fit_hw.forecast(steps=2)
```

2. Модель с фиксированными параметрами:

```
fit_hw_fixed = model_hw.fit(
    smoothing_level=0.1,
    smoothing_trend=0.9,
    smoothing_seasonal=0.4
)
```

Получить прогноз на заданный месяц.

5. Представление результатов

Отчёт должен содержать:

1. Цель работы.
2. Краткое описание исходных данных.
3. Основные этапы обработки данных.
4. Программный код построения моделей.
5. Полученные прогнозные значения.
6. Значение коэффициента детерминации (для линейной модели).
7. Графики временных рядов и прогнозов.
8. Выводы.

Форма представления отчёта:

- файл DOCX или PDF,
  или
- файл Jupyter Notebook (*.ipynb) с полностью воспроизводимыми результатами.

Лабораторная работа №3

Анализ социального графа и эмбеддинг узлов

1. Цель работы

Освоение методов анализа структуры социального графа, вычисления структурных метрик, выявления сообществ, построения эмбеддингов узлов и выполнения кластеризации на основе векторных представлений графа.

2. Исходные данные

Используется социальный граф Facebook из набора Stanford Network Analysis Project (SNAP).

Источник:
https://snap.stanford.edu/data/facebook_combined.txt.gz

Описание набора:

- 4039 узлов
- 88234 ребра
- неориентированный граф
- узлы — пользователи
- рёбра — дружеские связи

После скачивания файл необходимо:

1. Распаковать архив `.gz`
2. Загрузить файл `facebook_combined.txt`
3. Построить граф с использованием `networkx`

Пример загрузки:

```
import networkx as nx

G = nx.read_edgelist("facebook_combined.txt", nodetype=int)
```

Работа выполняется в Jupyter Notebook / Google Colab (Python).

3. Используемые библиотеки

```
import networkx as nx
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from karateclub import DeepWalk
from sklearn.cluster import DBSCAN, AffinityPropagation
from sklearn_extra.cluster import KMedoids
from sklearn.metrics import silhouette_score
from sklearn.metrics import adjusted_mutual_info_score, adjusted_rand_score
from sklearn.metrics.pairwise import cosine_distances
from sklearn.decomposition import PCA
```

4. Задание

4.1 Базовый анализ графа

1. Определить:
    - число узлов;
    - число рёбер;
    - плотность графа;
    - среднюю степень вершины;
    - диаметр графа (для наибольшей связной компоненты).
2. Рассчитать:
    - Degree centrality;
    - Betweenness centrality;
    - Eigenvector centrality;
    - PageRank.
3. Определить 5 узлов с наибольшим значением PageRank.

4.2 Выявление сообществ

1. Выполнить разбиение графа на сообщества (например, алгоритмом Louvain или greedy modularity).
2. Определить:
    - количество сообществ;
    - значение модульности.
3. Визуализировать граф с раскраской узлов по сообществам.
4. Сделать вывод о структуре графа.

4.3 Эмбеддинг узлов графа

1. Выполнить эмбеддинг графа методом DeepWalk.
2. Получить матрицу эмбеддингов.
3. Указать:
    - размерность вектора узла;
    - форму матрицы (число узлов × размерность).

Пример:

```
model = DeepWalk(dimensions=64)
model.fit(G)
embeddings = model.get_embedding()
```

4.4 Визуализация эмбеддингов

1. Применить PCA для снижения размерности до 2.
2. Построить двумерную визуализацию.
3. Определить визуально количество групп.

4.5 Поиск наиболее схожих узлов

1. Выбрать узел с максимальным PageRank.
2. Используя косинусное расстояние, определить 3 наиболее схожих узла.
3. Указать номера узлов.
4. Проинтерпретировать результат.

4.6 Кластеризация эмбеддингов

На основе эмбеддингов выполнить кластеризацию тремя методами:

1. DBSCAN (косинусное расстояние)
2. Affinity Propagation
3. K-Medoids

Для K-Medoids:

- подобрать число кластеров по максимальному значению коэффициента силуэта.

Для каждого метода определить:

- число кластеров;
- значение коэффициента силуэта.

4.7 Сравнение методов

1. Рассчитать:
   - Adjusted Mutual Information (AMI)
   - Adjusted Rand Index (ARI)
2. Определить:
   - какие два метода наиболее схожи по AMI;
   - какие два метода наиболее схожи по ARI.
3. Сделать вывод о согласованности методов кластеризации.

5. Требования к отчёту

Отчёт должен содержать:

1. Цель работы.
2. Описание структуры графа.
3. Рассчитанные метрики центральности.
4. Результаты выявления сообществ.
5. Размерность эмбеддингов.
6. Результаты кластеризации.
7. Значения коэффициента силуэта.
8. Значения AMI и ARI.
9. Интерпретацию найденных схожих узлов.
10. Выводы.

Форма представления:

- файл Jupyter Notebook (*.ipynb)
  или
- файл DOCX / PDF с кодом и результатами.

6. Контрольные вопросы

1. Чем отличаются структурные метрики графа от эмбеддингов узлов?
2. Почему DeepWalk позволяет выявлять скрытые сообщества?
3. В чём отличие плотности графа от модульности?
4. Почему косинусное расстояние часто используется для сравнения эмбеддингов?
5. В каких случаях DBSCAN предпочтительнее методов, требующих задания числа кластеров?

Лабораторная работа №4

Обучение нейронной сети и анализ переобучения (MNIST)

1. Цель работы

Освоение методов обучения полносвязной нейронной сети для задачи классификации изображений, анализ влияния оптимизаторов и функций активации на процесс обучения, а также экспериментальное исследование явления переобучения.

2. Исходные данные

Используется датасет **MNIST — база изображений рукописных цифр**.

Источник:
http://yann.lecun.com/exdb/mnist/

Загрузка выполняется автоматически через Keras:

```
from tensorflow.keras.datasets import mnist
```

Описание набора:

- 60000 изображений для обучения
- 10000 изображений для тестирования
- размер изображения: 28×28 пикселей
- 10 классов (цифры 0–9)

Работа выполняется в **Google Colab или Kaggle (Python, TensorFlow/Keras)**.

3. Используемые библиотеки

```
import numpy as np
import matplotlib.pyplot as plt

from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
from tensorflow.keras.optimizers import Adam, SGD, RMSprop
from tensorflow.keras.utils import to_categorical
```

4. Задание

4.1 Подготовка данных

1. Загрузить набор MNIST.
2. Нормализовать значения пикселей (деление на 255).
3. Преобразовать целевые метки в формат one-hot encoding.
4. Разделить данные на обучающую и тестовую выборки.

4.2 Построение базовой модели

Построить полносвязную нейронную сеть следующей архитектуры:

- Flatten
- Dense (128)
- Dense (64)
- Dense (10, softmax)

Использовать оптимизатор Adam (базовый эксперимент).

4.3 Метрики качества

Необходимо использовать:

- Accuracy
- Loss
- (при желании — Precision, Recall, F1)

Построить графики:

- зависимости функции потерь от эпохи;
- зависимости точности от эпохи.

4.4 Исследование влияния оптимизатора

Обучить модель, используя:

1. Adam
2. SGD
3. RMSprop

Сравнить:

- скорость сходимости;
- финальную точность;
- устойчивость обучения.

4.5 Исследование влияния функции активации

Провести эксперименты минимум с тремя функциями активации:

1. ReLU
2. Sigmoid
3. Tanh

Сравнить:

- скорость обучения;
- устойчивость;
- итоговое качество классификации.

4.6 Исследование переобучения

Используя функцию активации ReLU:

1. Обучить модель при:
    - 500 эпохах
    - 700 эпохах
    - 1000 эпохах
2. Построить графики:
    - train loss
    - validation loss
    - train accuracy
    - validation accuracy
3. Наглядно продемонстрировать явление переобучения.
4. Объяснить причину его возникновения.

4.7 Выбор оптимального числа эпох

1. Проанализировать графики.
2. Определить оптимальное число эпох.
3. Обосновать выбор.

5. Требования к отчёту

Отчёт должен содержать:

1. Цель работы.
2. Описание архитектуры нейронной сети.
3. Используемые метрики.
4. Графики обучения.
5. Сравнительную таблицу результатов:
    - оптимизаторы;
    - функции активации;
    - количество эпох.
6. Анализ переобучения.
7. Выводы.

Форма представления:

- файл Jupyter Notebook (*.ipynb)
  или
- файл DOCX / PDF с кодом и графиками.

6. Контрольные вопросы

1. В чём различие между функциями активации ReLU и Sigmoid?
2. Почему оптимизатор SGD может сходиться медленнее Adam?
3. Что такое переобучение?
4. Как по графикам определить момент начала переобучения?
5. Какие методы позволяют бороться с переобучением?

Лабораторная работа №5

Разметка изображений и формирование обучающего набора для задачи детекции объектов

1. Цель работы

Освоение методов разметки изображений для задач детекции объектов, формирования обучающих выборок и анализа влияния качества разметки на точность работы нейронной сети.

2. Исходные данные

Используется открытый датасет изображений людей в защитных касках:

Hard Hat Workers Dataset

Источник:
https://www.kaggle.com/datasets/andrewmvd/hard-hat-detection

Описание:

- изображения людей на строительных площадках
- классы:
  - helmet
  - head
  - person
- формат аннотаций: Pascal VOC (XML)

Для выполнения лабораторной работы:

1. Скачать датасет с Kaggle.
2. Выбрать 50–100 изображений для самостоятельной разметки.
3. Выполнить разметку заново (без использования готовых XML).

3. Используемые инструменты

Разметка

- LabelImg
  https://github.com/tzutalin/labelImg

Формат разметки:

- YOLO
  или
- Pascal VOC (по выбору преподавателя)

Обучение модели

Используется YOLOv8 (Ultralytics).

```
!pip install ultralytics
```

4. Задание

4.1 Теоретическая подготовка

1. Изучить метрики качества детекции:
    - Precision
    - Recall
    - mAP
    - IoU
2. Изучить форматы хранения разметки:
    - YOLO
    - Pascal VOC
    - COCO

4.2 Разметка изображений

1. Выполнить разметку выбранных изображений в LabelImg.
2. Сохранить аннотации в формате YOLO.
3. Проверить корректность координат bounding box.
4. Разделить данные на:
    - train
    - val
    - test

4.3 Формирование структуры датасета

Подготовить структуру:

```
dataset/
  ├── images/
  │     ├── train/
  │     ├── val/
  │     └── test/
  └── labels/
        ├── train/
        ├── val/
        └── test/
```

Создать файл `data.yaml` с описанием классов.

4.4 Обучение модели

Обучить YOLOv8 на размеченном датасете:

```
from ultralytics import YOLO

model = YOLO("yolov8n.pt")
model.train(data="data.yaml", epochs=50)
```

4.5 Оценка качества

1. Получить значения:
    - mAP@0.5
    - mAP@0.5:0.95
    - Precision
    - Recall
2. Построить графики обучения.
3. Выполнить инференс на тестовых изображениях.
4. Проанализировать ошибки детекции.

4.6 Анализ влияния качества разметки

1. Намеренно допустить ошибки в разметке (частично).
2. Повторить обучение.
3. Сравнить метрики.
4. Объяснить различия.

5. Требования к отчёту

Отчёт должен содержать:

1. Цель работы.
2. Краткое описание форматов разметки.
3. Примеры размеченных изображений.
4. Структуру сформированного датасета.
5. Графики обучения до корректировки разметки.
6. Графики обучения после корректировки.
7. Таблицу значений метрик.
8. Выводы.

Форма представления:

- Jupyter Notebook (*.ipynb)
  или
- DOCX / PDF с кодом и изображениями.

6. Контрольные вопросы

1. Что такое IoU?
2. Чем отличается mAP@0.5 от mAP@0.5:0.95?
3. Почему качество разметки влияет на устойчивость модели?
4. В чём отличие форматов YOLO и COCO?
5. Какие ошибки разметки наиболее критичны для обучения?

Лабораторная работа №6

Установление авторства текста методами машинного обучения

1. Цель работы

Освоение методов предобработки текстовых данных, формирования признакового пространства и построения моделей машинного обучения для решения задачи автоматического установления авторства литературных произведений.

2. Исходные данные

Для выполнения работы используются материалы:

- архив `authors.zip`, содержащий тексты различных авторов;
- папки с текстами для определения авторства (например `5/blok_a_a`);

Архив содержит:

- обучающие тексты, сгруппированные по авторам;
- набор текстов с неизвестным авторством для классификации.

Перед началом работы необходимо:

1. Скачать архив `authors.zip`.
2. Распаковать его в рабочую директорию Jupyter Notebook / Google Colab.
3. Проверить структуру папок.

Работа выполняется в среде Python.

3. Используемые библиотеки

```
import os
import glob
import zipfile
import numpy as np
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import classification_report
from sklearn.metrics import accuracy_score
```

4. Задание

4.1 Теоретическая подготовка

Изучить:

- методы токенизации текста;
- подход Bag-of-Words;
- TF-IDF;

- n-граммы;
- вероятностные и линейные модели классификации текстов.

4.2 Предобработка текста

1. Прочитать тексты из папок авторов.
2. Привести текст к нижнему регистру.
3. Удалить знаки препинания.
4. Сформировать список текстов и соответствующих меток авторов.
5. Разделить данные на обучающую и тестовую выборки.

4.3 Формирование признаков

Построить матрицу признаков двумя способами:

1. Bag-of-Words (CountVectorizer);
2. TF-IDF (TfidfVectorizer).

Пример:

```
vectorizer = TfidfVectorizer(
    ngram_range=(1,2),
    max_features=5000
)
X = vectorizer.fit_transform(texts)
```

4.4 Построение базовой модели

Использовать один из методов:

- Logistic Regression
  или
- Multinomial Naive Bayes

Обучить модель на обучающей выборке.

Оценить качество на тестовой выборке:

- accuracy
- precision
- recall
- f1-score

4.5 Определение авторства неизвестных текстов

1. Прочитать тексты из папки `blok_a_a`.
2. Преобразовать их тем же векторизатором.
3. Получить вероятности принадлежности к каждому автору.
4. Представить результаты в виде таблицы:

Текст	Автор 1	Автор 2	Автор 3	...

Указать наиболее вероятного автора для каждого текста.

4.6 Альтернативный метод

Повторить эксперимент, изменив:

- алгоритм классификации
  или
- способ векторизации.

Сравнить:

- точность;
- распределение вероятностей;
- устойчивость результатов.

4.7 Анализ результатов

1. Сравнить основной и альтернативный методы.
2. Оценить, насколько уверенно модель определяет авторство.
3. Сделать вывод о различимости авторских стилей.

5. Требования к отчёту

Отчёт должен содержать:

1. Цель работы.
2. Описание структуры данных.
3. Описание метода векторизации.
4. Результаты классификации (метрики).
5. Таблицу вероятностей для неизвестных текстов.
6. Сравнение двух методов.
7. Выводы.

Форма представления:

- файл Jupyter Notebook (*.ipynb)
  или
- файл DOCX / PDF с кодом и таблицами.

6. Контрольные вопросы

1. В чём различие между Bag-of-Words и TF-IDF?
2. Почему вероятностные методы хорошо подходят для анализа текста?
3. Как влияет размер словаря на качество модели?
4. Что такое n-граммы и зачем они используются?
5. Почему задача установления авторства является задачей классификации?

Лабораторная работа №7

Выбор оптимального метода анализа данных и построение ансамбля моделей

1. Цель работы

Освоение методов сравнительного анализа алгоритмов машинного обучения, выбора оптимальной модели для решения задачи классификации и построения ансамбля моделей с использованием методов стекинга.

2. Исходные данные

Используется датасет Adult Income (Census Income Dataset).

Источник:
https://archive.ics.uci.edu/dataset/2/adult
или
https://www.kaggle.com/datasets/uciml/adult-census-income

Описание задачи:

По социально-демографическим характеристикам человека определить, превышает ли его доход 50 000 долларов в год.

Тип задачи: бинарная классификация.

3. Используемые библиотеки

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.ensemble import StackingClassifier

from sklearn.metrics import accuracy_score
from sklearn.metrics import classification_report
```

4. Задание

4.1 Предобработка данных

1. Загрузить датасет.
2. Удалить или корректно обработать пропуски.
3. Разделить признаки на:
    - числовые;

    - категориальные.
4. Выполнить:
    - нормализацию числовых признаков;
    - кодирование категориальных признаков (One-Hot Encoding).
5. Разделить данные на обучающую и тестовую выборки.

4.2 Анализ отдельных методов

Провести обучение и анализ не менее 5 методов машинного обучения:

1. Logistic Regression
2. KNN
3. Decision Tree
4. Random Forest
5. Gradient Boosting

Для каждого метода:

- выполнить настройку гиперпараметров;
- обучить модель;
- оценить качество (accuracy, precision, recall, f1-score);
- сравнить результаты.

4.3 Выбор трёх оптимальных моделей

На основе анализа выбрать 3 наиболее эффективные модели.

Обосновать выбор.

4.4 Формирование ансамбля (Stacking)

Сформировать ансамбль из выбранных моделей:

```
estimators = [
    ('rf', RandomForestClassifier()),
    ('gb', GradientBoostingClassifier()),
    ('lr', LogisticRegression())
]

stack = StackingClassifier(
    estimators=estimators,
    final_estimator=LogisticRegression()
)
```

Обучить ансамбль и оценить его качество.

4.5 Сравнение результатов

Сравнить:

- точность лучших одиночных моделей;
- точность ансамбля;

- устойчивость результатов.

Проанализировать:

- улучшилось ли качество;
- в каких случаях использование ансамблей оправдано.

5. Требования к отчёту

Отчёт должен содержать:

1. Цель работы.
2. Описание датасета.
3. Описание процедуры предобработки.
4. Таблицу сравнения методов.
5. Описание выбранных трёх моделей.
6. Результаты работы ансамбля.
7. Сравнительный анализ.
8. Выводы.

Форма представления:

- Jupyter Notebook (*.ipynb)
  или
- DOCX / PDF с кодом и таблицами.

6. Контрольные вопросы

1. В чём отличие ансамблей от одиночных моделей?
2. Почему стекинг может давать более высокую точность?
3. В каких случаях ансамбль не улучшает результат?
4. Чем отличается Random Forest от Gradient Boosting?
5. Почему важна корректная предобработка данных при сравнении моделей?